Deep Learning - Gradient Descent - Linear regression

Refai Arafat Rossi

October 2026

1 Introduction

[aspectratio=169, 10pt]beamer

Boadilla whale

[utf8]inputenc amsmath, amssymb, amsfonts tikz shapes.geometric, arrows.meta, positioning, calc booktabs multicol

navigation symbols footline[frame number]

[Chapter 5.1.4: Linear Regression]Linear Regression: Features, Weights, Bias & MSE Based on Chapter 5.1.4, *Deep Learning* (Ian Goodfellow et al.) Master's Level Lecture Deck Department of Computer Science / Data Science October 10, 2026

height

1. Problem: Predict Deep Learning Model Training Time Suppose we want to estimate how many minutes a model will take to train based on two observed features:

- x_1: Number of training epochs
- x_2: Batch size measured in units of 16 samples

We decide to model the relationship using a **Linear Regression Model**:

$$\hat{y} = w_1 x_1 + w_2 x_2 + b$$

where $\hat{y}$ is predicted training time (minutes), w_1, w_2 are weights, and b is the bias.

Learned Model Example Suppose our algorithm has learned parameters $w_1 = 2, w_2 = 3, b = 5$:

$$\hat{y} = 2x_1 + 3x_2 + 5$$

2. What Is the Machine Learning Task? A machine learning task specifies what the model is supposed to do. Here, the task is mapping inputs to continuous real values:

$$\mathbf{x} \longrightarrow \hat{y} \in R$$

0.5 Regression Task ($\hat{y} \in R$) Output is a continuous variable:

- Model training time (minutes)
- Housing prices ($)
- GPU operating temperature ($^{\circ}C$)

0.5 Classification Task ($\hat{y} \in \{0, 1, \ldots\}$) Output is a discrete categorical label:

- Image identification (Cat vs. Dog)
- Medical diagnosis (Benign vs. Malignant)

3 & 4. Features and Targets Feature ($\mathbf{x}$) An input variable/attribute passed to the model to aid prediction.

$$x_1 = numberofepochs, \quad x_2 = batch - sizeunits$$

Target (y) The true ground-truth value we aim to predict:

$$y = actualobservedtrainingtime(minutes)$$

Example Instance: If an experiment uses $x_1 = 10$ epochs and $x_2 = 4$ batch units with ground truth $y = 37$ minutes:

$\hat{y} = 2(10)+3(4)+5 = 20+12+5 = \mathbf{37} Error(y-\hat{y}) = 37-37 = 0 \quad (Perfectprediction!)$

5. What Is a Feature Vector? Instead of handling variables individually, we package inputs into a **feature vector x**:

$$\mathbf{x} = x_1 x_2 = 104$$

General n-dimensional Feature Vector For an observation with n input features:

$$\mathbf{x} = x_1 x_2 \vdots x_n \in R^n$$

Each sample in a dataset is represented mathematically as a point in n-dimensional vector space.

6 & 7. Weights and Feature Interpretation Model equation: $\hat{y} = 2x_1 + 3x_2 + 5w_1 = 2$, $w_2 = 3$.

Linear Model Weight Meaning A weight w_j represents the expected change in output $\hat{y}$ per unit change in x_j, holding all other features fixed:

- $w_1 = 2 + 1$ epoch increases predicted time by 2 minutes.
- $w_2 = 3 + 1$ batch-size unit increases predicted time by 3 minutes.

Important Nuance: Weight Magnitude vs. Feature Scale A larger weight does NOT automatically imply a more important feature! Scale matters (e.g., $x_1 = ageinyears$ vs. $x_2 = incomeindollars$). Feature normalization is required before comparing weight magnitudes directly.

8 & 9. Parameters vs. Weights 0.5 Model Parameters (θ) The set of all learnable values adjusted by training algorithms:

$$\theta = \{w_1, w_2, \ldots, w_n, b\}$$

0.5 Structural Decomposition

- **Weights (w_j):** Parameters multiplying features or activations.
- **Bias (b):** Additive parameter shifting output independent of inputs.

$$Weightsandbiasesarespecificsubsetsofmodelparameters\theta.$$

10. Vector Notation for Linear Regression We write linear regression using vector dot products:

$$\mathbf{w} = w_1 w_2, \quad \mathbf{x} = x_1 x_2 \mathbf{w}^T \mathbf{x} = w_1 w_2 x_1 x_2 = w_1 x_1 + w_2 x_2$$

Vector Formulation of Linear Regression

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$

This linear combination is the exact foundational building block used in deep neural network layers.

11 & 12. Linear Regression as an Artificial Neuron 0.5 **Structural Comparison:**

- **Linear Regression:**
$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$
- **Artificial Neuron:**
$$z = \mathbf{w}^T \mathbf{x} + b, \quad a = f(z)$$

Linear regression is simply a single neuron with an identity activation function ($f(z) = z$).

0.5 [x=1.5cm, y=1.2cm, ¿=stealth] [circle, draw, fill=blue!10] (x1) at (0, 0.8) x_1; [circle, draw, fill=blue!10] (x2) at (0, -0.8) x_2; [circle, draw, fill=green!10, inner sep=3pt] (sum) at (1.5, 0) $\sum$; [draw, fill=yellow!10, minimum height=0.6cm] (bias) at (1.5, -1.2) $+b$; [circle, draw, fill=red!10] (y) at (2.8, 0) $\hat{y}$;

Exp	Epochs (x_1)	Batch (x_2)	Actual (y)	Predicted ($\hat{y}$)	Error ($y - \hat{y}$)	Squared Erro
A	2	2	15	$2(2) + 3(2) + 5 = 15$	0	0
B	4	3	22	$2(4) + 3(3) + 5 = 22$	0	0
C	6	4	29	$2(6) + 3(4) + 5 = 29$	0	0
D	8	5	35	$2(8) + 3(5) + 5 = \mathbf{36}$	$\mathbf{-1}$	**1**

[-¿] (x1) – node[above, xshift=-2pt] $\times w_1$ (sum); [-¿] (x2) – node[below, xshift=-2pt] $\times w_2$ (sum); [-¿] (bias) – (sum); [-¿] (sum) – (y);

13–17. Dataset Evaluation Example Consider model $\hat{y} = 2x_1 + 3x_2 + 5$ evaluated on 4 experimental runs:

Exp D exhibits a prediction discrepancy ($\hat{y} = 36$ vs. $y = 35$). We need a quantitative metric to measure overall dataset error.

18–21. Performance Metric: Mean Squared Error (MSE) To quantify model error across m dataset samples, we compute the Mean Squared Error:

$$MSE = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

0.5 **Why square the residuals?**

- Prevents positive and negative errors from canceling out ($+2 + (-2) = 0$).
- Penalizes larger deviations much more severely ($1^2 = 1$ vs. $5^2 = 25$).

0.5 MSE Calculation for Dataset

$$MSE = \frac{0 + 0 + 0 + 1}{4} = \mathbf{0.25}$$

An MSE of 0 represents perfect prediction across all samples.

25 & 26. Residuals and Geometric Interpretation Residual Definition The residual $r^{(i)}$ for sample i is the vertical distance between the actual target value $y^{(i)}$ and the predicted value $\hat{y}^{(i)}$:

$$r^{(i)} = y^{(i)} - \hat{y}^{(i)}$$

0.5 [scale=0.65] [-¿] (0,0) – (5,0) node[right] x; [-¿] (0,0) – (0,5) node[above] y; [thick, blue] (0,1) – (4.5,4.5) node[right] $\hat{y} = wx + b$; [red] (3, 2.3) circle (3pt) node[right] $(x^{(i)}, y^{(i)})$; [dashed, red, thick] (3, 2.3) – (3, 3.33); [left, red] at (3, 2.8) $r^{(i)}$;

0.5 **Key Concept:** Linear regression finds parameters w, b that minimize the average sum of these squared vertical distances across all data points.

27–30. Geometric Meaning and Necessity of Bias Why do we need an explicit bias term b?

0.5 Without Bias ($b = 0$): $\hat{y} = wx$ Forces the line/plane through the origin $(0, 0)$. If target $y = 5$ when $x = 0$, a bias-free model fails completely.

0.5 With Bias ($b \neq 0$): $\hat{y} = wx + b$ Allows affine transformations. Bias shifts the decision boundary / regression hyperplane up or down.

Term	Meaning in Linear Regression Context
Task	Predict training runtime given model hyperparameters.
Feature	Individual input attribute (x_1 = epochs, x_2 = batch units).
Feature Vector (x)	Vector packaging all input variables for a single sample.
Target (y)	Ground-truth true real-world observation.
Prediction ($\hat{y}$)	Output generated by model $\mathbf{w}^T\mathbf{x} + b$.
Weight (w)	Multiplicative parameters defining feature contributions.
Bias (b)	Additive parameter enabling affine shift of predictions.
Parameters (θ)	Set of all learnable variables ($\theta = \{\mathbf{w}, b\}$).
Residual (r)	Raw prediction error ($y - \hat{y}$).
MSE	Mean Squared Error measuring performance over dataset.

- **1D Line ($\hat{y} = wx + b$):** w dictates slope; b dictates y-intercept.
- **2D Plane ($\hat{y} = w_1x_1 + w_2x_2 + b$):** w_1, w_2 dictate orientation; b shifts plane elevation.

32 & 33. Framing Parameter Learning via Optimization We don't know ideal weights w and bias b beforehand. We start with random parameter initializations and optimize via **Gradient Descent**.

Loss Function to Minimize:

$$J(w_1, w_2, b) = \frac{1}{m}\sum_{i=1}^{m}\left((w_1x_1^{(i)} + w_2x_2^{(i)} + b) - y^{(i)}\right)^2$$

Gradient Updates:

$$w_j \leftarrow w_j - \eta\frac{\partial J}{\partial w_j}, \qquad b \leftarrow b - \eta\frac{\partial J}{\partial b}$$

$$\text{Input x} \longrightarrow Prediction\hat{y} \longrightarrow MSELossJ \longrightarrow Gradients\nabla J \longrightarrow ParameterUpdates$$

35. Terminology Summary Table

36 & 37. Student Classroom Exercises Exercise 1: Zero Error Calculation Given model $\hat{y} = 3x_1 + 2x_2 + 4$. For sample $\mathbf{x} = [5, 3]^T$ with actual target $y = 25$. Compute prediction, error, and squared error.

Solution:

$$\hat{y} = 3(5) + 2(3) + 4 = 15 + 6 + 4 = \mathbf{25}e = 25 - 25 = \mathbf{0}e^2 = \mathbf{0}$$

Exercise 2: Non-Zero Error Calculation For the same model, given sample $\mathbf{x} = [6, 4]^T$ with actual target $y = 32$.

Solution:

$$\hat{y} = 3(6) + 2(4) + 4 = 18 + 8 + 4 = \mathbf{30}e = 32 - 30 = \mathbf{2}e^2 = \mathbf{4}$$

38. Core Equations to Remember

1. **Linear Regression Prediction Equation:**

$$\underbrace{\hat{y}}_{Prediction} = \underbrace{\mathbf{w}^T}_{Weights}\underbrace{\mathbf{x}}_{Features} + \underbrace{b}_{Bias}$$

2. **Mean Squared Error Objective Function:**

$$MSE = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

3. **Optimization Goal:**

$$\mathbf{w}^*, b^* = \arg\min_{\mathbf{w},b} MSE(\mathbf{w}, b)$$

39. Final Summary: Linear Model → Deep Neural Network

Progression of Model Complexity:

1. **Linear Regression (Single Identity Neuron):**

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$

2. **Single Non-Linear Neuron:**

$$a = f(\mathbf{w}^T \mathbf{x} + b)$$

3. **Multi-Layer Deep Neural Network:**

$$\mathbf{h}^{(1)} = f_1(W^{(1)}\mathbf{x}+\mathbf{b}^{(1)}), \quad \mathbf{h}^{(2)} = f_2(W^{(2)}\mathbf{h}^{(1)}+\mathbf{b}^{(2)}), \quad \ldots, \quad \hat{y} = f_L(W^{(L)}\mathbf{h}^{(L-1)}+\mathbf{b}^{(L)})$$

Final Conceptual Takeaway Deep networks are constructed by stacking the fundamental linear regression operation ($\mathbf{w}^T \mathbf{x} + b$) across multiple layers interleaved with non-linear activation functions.

Deep Learning - Linear Regression: Features, Weights, Bias, MSE

Refai Arafat Rossi

October 2026

1 Introduction

[Chapter 5.1.4: Linear Regression]Linear Regression: Features, Weights, Bias & MSE Based on Chapter 5.1.4, *Deep Learning* (Ian Goodfellow et al.) Master's Level Lecture Deck Department of Computer Science / Data Science October 10, 2026

height

1. Problem: Predict Deep Learning Model Training Time Suppose we want to estimate how many minutes a model will take to train based on two observed features:

- x_1: Number of training epochs
- x_2: Batch size measured in units of 16 samples

We decide to model the relationship using a **Linear Regression Model**:

$$\hat{y} = w_1 x_1 + w_2 x_2 + b$$

where $\hat{y}$ is predicted training time (minutes), w_1, w_2 are weights, and b is the bias.

Learned Model Example Suppose our algorithm has learned parameters $w_1 = 2, w_2 = 3, b = 5$:

$$\hat{y} = 2x_1 + 3x_2 + 5$$

2. What Is the Machine Learning Task? A machine learning task specifies what the model is supposed to do. Here, the task is mapping inputs to continuous real values:

$$\mathbf{x} \longrightarrow \hat{y} \in R$$

0.5 Regression Task ($\hat{y} \in R$) Output is a continuous variable:

- Model training time (minutes)

- Housing prices ($)
- GPU operating temperature ($^{\circ}C$)

0.5 Classification Task ($\hat{y} \in \{0, 1, \ldots\}$) Output is a discrete categorical label:

- Image identification (Cat vs. Dog)
- Medical diagnosis (Benign vs. Malignant)

3 & 4. Features and Targets Feature ($\mathbf{x}$) An input variable/attribute passed to the model to aid prediction.

$$x_1 = numberofepochs, \quad x_2 = batch - sizeunits$$

Target (y) The true ground-truth value we aim to predict:

$$y = actualobservedtrainingtime(minutes)$$

Example Instance: If an experiment uses $x_1 = 10$ epochs and $x_2 = 4$ batch units with ground truth $y = 37$ minutes:

$$\hat{y} = 2(10)+3(4)+5 = 20+12+5 = \mathbf{37} Error(y-\hat{y}) = 37-37 = 0 \quad (Perfectprediction!)$$

5. What Is a Feature Vector? Instead of handling variables individually, we package inputs into a **feature vector x**:

$$\mathbf{x} = x_1 x_2 = 104$$

General n-dimensional Feature Vector For an observation with n input features:

$$\mathbf{x} = x_1 x_2 \vdots x_n \in R^n$$

Each sample in a dataset is represented mathematically as a point in n-dimensional vector space.

6 & 7. Weights and Feature Interpretation Model equation: $\hat{y} = 2x_1 + 3x_2 + 5w_1 = 2$, $w_2 = 3$.

Linear Model Weight Meaning A weight w_j represents the expected change in output $\hat{y}$ per unit change in x_j, holding all other features fixed:

- $w_1 = 2 + 1$ epoch increases predicted time by 2 minutes.
- $w_2 = 3 + 1$ batch-size unit increases predicted time by 3 minutes.

Important Nuance: Weight Magnitude vs. Feature Scale A larger weight does NOT automatically imply a more important feature! Scale matters (e.g., $x_1 = ageinyears$ vs. $x_2 = incomeindollars$). Feature normalization is required before comparing weight magnitudes directly.

8 & 9. Parameters vs. Weights 0.5 Model Parameters (θ) The set of all learnable values adjusted by training algorithms:

$$\theta = \{w_1, w_2, \ldots, w_n, b\}$$

0.5 Structural Decomposition

- **Weights (w_j):** Parameters multiplying features or activations.
- **Bias (b):** Additive parameter shifting output independent of inputs.

$$Weights and biases are specific subsets of model parameters \theta.$$

10. Vector Notation for Linear Regression We write linear regression using vector dot products:

$$\mathbf{w} = w_1 w_2, \quad \mathbf{x} = x_1 x_2 \mathbf{w}^T \mathbf{x} = w_1 w_2 x_1 x_2 = w_1 x_1 + w_2 x_2$$

Vector Formulation of Linear Regression

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$

This linear combination is the exact foundational building block used in deep neural network layers.

11 & 12. Linear Regression as an Artificial Neuron 0.5 **Structural Comparison:**

- **Linear Regression:**
$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$
- **Artificial Neuron:**
$$z = \mathbf{w}^T \mathbf{x} + b, \quad a = f(z)$$

Linear regression is simply a single neuron with an identity activation function ($f(z) = z$).

0.5 [x=1.5cm, y=1.2cm, ¿=stealth] [circle, draw, fill=blue!10] (x1) at (0, 0.8) x_1; [circle, draw, fill=blue!10] (x2) at (0, -0.8) x_2; [circle, draw, fill=green!10, inner sep=3pt] (sum) at (1.5, 0) $\sum$; [draw, fill=yellow!10, minimum height=0.6cm] (bias) at (1.5, -1.2) $+b$; [circle, draw, fill=red!10] (y) at (2.8, 0) $\hat{y}$; [-¿] (x1) – node[above, xshift=-2pt] $\times w_1$ (sum); [-¿] (x2) – node[below, xshift=-2pt] $\times w_2$ (sum); [-¿] (bias) – (sum); [-¿] (sum) – (y);

13–17. Dataset Evaluation Example Consider model $\hat{y} = 2x_1 + 3x_2 + 5$ evaluated on 4 experimental runs:

Exp D exhibits a prediction discrepancy ($\hat{y} = 36$ vs. $y = 35$). We need a quantitative metric to measure overall dataset error.

18–21. Performance Metric: Mean Squared Error (MSE) To quantify model error across m dataset samples, we compute the Mean Squared Error:

$$MSE = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

0.5 **Why square the residuals?**

Exp	Epochs (x_1)	Batch (x_2)	Actual (y)	Predicted ($\hat{y}$)	Error ($y - \hat{y}$)	Squared Erro
A	2	2	15	$2(2) + 3(2) + 5 = 15$	0	0
B	4	3	22	$2(4) + 3(3) + 5 = 22$	0	0
C	6	4	29	$2(6) + 3(4) + 5 = 29$	0	0
D	8	5	35	$2(8) + 3(5) + 5 = \mathbf{36}$	$\mathbf{-1}$	**1**

- Prevents positive and negative errors from canceling out ($+2 + (-2) = 0$).
- Penalizes larger deviations much more severely ($1^2 = 1$ vs. $5^2 = 25$).

0.5 MSE Calculation for Dataset

$$MSE = \frac{0 + 0 + 0 + 1}{4} = \mathbf{0.25}$$

An MSE of 0 represents perfect prediction across all samples.

25 & 26. Residuals and Geometric Interpretation Residual Definition The residual $r^{(i)}$ for sample i is the vertical distance between the actual target value $y^{(i)}$ and the predicted value $\hat{y}^{(i)}$:

$$r^{(i)} = y^{(i)} - \hat{y}^{(i)}$$

0.5 [scale=0.65] [-¿] (0,0) – (5,0) node[right] x; [-¿] (0,0) – (0,5) node[above] y; [thick, blue] (0,1) – (4.5,4.5) node[right] $\hat{y} = wx + b$; [red] (3, 2.3) circle (3pt) node[right] $(x^{(i)}, y^{(i)})$; [dashed, red, thick] (3, 2.3) – (3, 3.33); [left, red] at (3, 2.8) $r^{(i)}$;

0.5 **Key Concept:** Linear regression finds parameters w, b that minimize the average sum of these squared vertical distances across all data points.

27–30. Geometric Meaning and Necessity of Bias Why do we need an explicit bias term b?

0.5 Without Bias ($b = 0$): $\hat{y} = wx$ Forces the line/plane through the origin $(0, 0)$. If target $y = 5$ when $x = 0$, a bias-free model fails completely.

0.5 With Bias ($b \neq 0$): $\hat{y} = wx + b$ Allows affine transformations. Bias shifts the decision boundary / regression hyperplane up or down.

- **1D Line ($\hat{y} = wx + b$):** w dictates slope; b dictates y-intercept.
- **2D Plane ($\hat{y} = w_1x_1 + w_2x_2 + b$):** w_1, w_2 dictate orientation; b shifts plane elevation.

32 & 33. Framing Parameter Learning via Optimization We don't know ideal weights w and bias b beforehand. We start with random parameter initializations and optimize via **Gradient Descent**.

Loss Function to Minimize:

$$J(w_1, w_2, b) = \frac{1}{m} \sum_{i=1}^{m} \left((w_1 x_1^{(i)} + w_2 x_2^{(i)} + b) - y^{(i)} \right)^2$$

Term	Meaning in Linear Regression Context
Task	Predict training runtime given model hyperparameters.
Feature	Individual input attribute (x_1 = epochs, x_2 = batch units).
Feature Vector (x)	Vector packaging all input variables for a single sample.
Target (y)	Ground-truth true real-world observation.
Prediction ($\hat{y}$)	Output generated by model $\mathbf{w}^T\mathbf{x} + b$.
Weight (w)	Multiplicative parameters defining feature contributions.
Bias (b)	Additive parameter enabling affine shift of predictions.
Parameters (θ)	Set of all learnable variables ($\theta = \{\mathbf{w}, b\}$).
Residual (r)	Raw prediction error ($y - \hat{y}$).
MSE	Mean Squared Error measuring performance over dataset.

Gradient Updates:

$$w_j \leftarrow w_j - \eta \frac{\partial J}{\partial w_j}, \qquad b \leftarrow b - \eta \frac{\partial J}{\partial b}$$

$$\text{Input x} \longrightarrow Prediction\hat{y} \longrightarrow MSELossJ \longrightarrow Gradients\nabla J \longrightarrow ParameterUpdates$$

35. Terminology Summary Table

36 & 37. Student Classroom Exercises Exercise 1: Zero Error Calculation Given model $\hat{y} = 3x_1 + 2x_2 + 4$. For sample $\mathbf{x} = [5, 3]^T$ with actual target $y = 25$. Compute prediction, error, and squared error.

Solution:

$$\hat{y} = 3(5) + 2(3) + 4 = 15 + 6 + 4 = \mathbf{25} e = 25 - 25 = \mathbf{0} e^2 = \mathbf{0}$$

Exercise 2: Non-Zero Error Calculation For the same model, given sample $\mathbf{x} = [6, 4]^T$ with actual target $y = 32$.

Solution:

$$\hat{y} = 3(6) + 2(4) + 4 = 18 + 8 + 4 = \mathbf{30} e = 32 - 30 = \mathbf{2} e^2 = \mathbf{4}$$

38. Core Equations to Remember

1. **Linear Regression Prediction Equation:**

$$\underbrace{\hat{y}}_{Prediction} = \underbrace{\mathbf{w}^T}_{Weights} \underbrace{\mathbf{x}}_{Features} + \underbrace{b}_{Bias}$$

2. **Mean Squared Error Objective Function:**

$$MSE = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

3. **Optimization Goal:**

$$\mathbf{w}^*, b^* = \arg\min_{\mathbf{w}, b} MSE(\mathbf{w}, b)$$

39. Final Summary: Linear Model → Deep Neural Network

Progression of Model Complexity:

1. **Linear Regression (Single Identity Neuron):**

$$\hat{y} = \mathbf{w}^T\mathbf{x} + b$$

2. **Single Non-Linear Neuron:**

$$a = f(\mathbf{w}^T\mathbf{x} + b)$$

3. **Multi-Layer Deep Neural Network:**

$$\mathbf{h}^{(1)} = f_1(W^{(1)}\mathbf{x}+\mathbf{b}^{(1)}), \quad \mathbf{h}^{(2)} = f_2(W^{(2)}\mathbf{h}^{(1)}+\mathbf{b}^{(2)}), \quad \ldots, \quad \hat{y} = f_L(W^{(L)}\mathbf{h}^{(L-1)}+\mathbf{b}^{(L)})$$

Final Conceptual Takeaway Deep networks are constructed by stacking the fundamental linear regression operation ($\mathbf{w}^T\mathbf{x} + b$) across multiple layers interleaved with non-linear activation functions.

Deep Learning - Activation Functions - Nonlinear Learning

Refai Arafat Rossi

October 2026

1 Introduction

[Chapter 6: Nonlinear Learning]Nonlinear Learning From Activation Functions to Backpropagation Master's Level Lecture Deck Department of Computer Science / Data Science October 10, 2026

---

height

1. Problem: Electricity Demand vs. Temperature Imagine an electricity grid operator planning daily power generation.

- **Cold days:** Demand rises due to electric space heating.
- **Hot days:** Demand rises due to air conditioning.
- **Comfortable days:** Demand is lowest around moderate temperatures.

Mathematical Formulation

- Input x: Temperature deviation from comfortable baseline.
- Target y: Additional electricity demand.
- Illustrative Ground Truth Model:

$$y = x^2$$

Sample Data Points:

Temp Deviation (x)	-2	-1	0	1	2
Demand ($y = x^2$)	4	1	0	1	4

2. Why Do We Need a Neural Network? 0.5 **Model A: Linear Regression**

$$\hat{y} = wx + b$$

Limitation Can only fit a straight line. Fails completely on U-shaped non-linear curves like $y = x^2$.

Model B: Single-Layer Neural Network

$$h = f(wx + b)$$

Non-linear activation $f(\cdot)$ combines piecewise signals to approximate curves.

0.5 **Model C: Deep Neural Network**

$$h^{(l)} = f\left(W^{(l)}h^{(l-1)} + b^{(l)}\right)$$

Multiple stacked layers construct hierarchical, highly complex functional representations.

Takeaway A linear model cannot capture parabolic demand. Hidden layers with non-linear activations provide universal approximation capabilities.

3. Activation Functions: ReLU and Sigmoid An artificial neuron calculates $z = wx + b$, then applies activation $a = f(z)$.

0.5 Rectified Linear Unit (ReLU)

$$ReLU(z) = \max(0, z)$$

- $ReLU(-3) = 0$
- $ReLU(2.5) = 2.5$
- Standard choice for hidden layers (simple derivative, avoids gradient saturation).

0.5 Sigmoid Activation

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- $\sigma(0) = 0.5$
- $\sigma(2) \approx 0.881, \quad \sigma(-2) \approx 0.119$
- Maps $R \to (0, 1)$. Standard choice for binary classification outputs.

4. Network Architecture for Electricity Demand 0.5 **1-Hidden-Layer Regression Network: Hidden 1:** $z_1 = w_1 x + b_1, \quad h_1 = ReLU(z_1)$
Hidden 2: $z_2 = w_2 x + b_2, \quad h_2 = ReLU(z_2)$
Output: $\hat{y} = v_1 h_1 + v_2 h_2 + c$

Vector Matrix Form

$$z^{(1)} = W^{(1)}x + b^{(1)}$$
$$h^{(1)} = ReLU\left(z^{(1)}\right)$$
$$\hat{y} = W^{(2)}h^{(1)} + b^{(2)}$$

0.5 [x=1.6cm, y=1.2cm, ¿=stealth] [circle, draw, fill=blue!10] (x) at (0, 0) x;
[circle, draw, fill=green!10] (h1) at (1.5, 0.8) h_1; [circle, draw, fill=green!10] (h2) at (1.5, -0.8) h_2;
[circle, draw, fill=red!10] (y) at (3, 0) $\hat{y}$;
[-¿] (x) – node[above left] w_1, b_1 (h1); [-¿] (x) – node[below left] w_2, b_2 (h2); [-¿] (h1) – node[above right] v_1 (y); [-¿] (h2) – node[below right] v_2, c (y);

5 & 6. Part B — Numerical Forward Pass Example **Training Instance:** $x = 1, \quad TrueTargety = 1.$

Initial Parameters:

$$w_1 = 1,\ b_1 = 0 \quad | \quad w_2 = 0.5,\ b_2 = 0.5 \quad | \quad v_1 = 0.2,\ v_2 = 0.2,\ c = 0$$

0.5 **Hidden Neuron 1:**

- $z_1 = (1)(1) + 0 = 1$
- $h_1 = ReLU(1) = \mathbf{1}$

Hidden Neuron 2:

- $z_2 = (0.5)(1) + 0.5 = 1$
- $h_2 = ReLU(1) = \mathbf{1}$

0.5 **Output Neuron ($\hat{y}$):**

$$\hat{y} = (0.2)(1) + (0.2)(1) + 0 = \mathbf{0.4}$$

Prediction Error

$$e = \hat{y} - y = 0.4 - 1.0 = \mathbf{-0.6}$$

The network underpredicted the true target (0.4 vs. 1.0).

7. Calculate the Loss To evaluate model error for differentiation, we use the **half-squared error** for a single sample:

$$L = \frac{1}{2}(\hat{y} - y)^2$$

Numerical Evaluation:

$$L = \frac{1}{2}(0.4 - 1.0)^2 = \frac{1}{2}(-0.6)^2 = \frac{1}{2}(0.36) = \mathbf{0.18}$$

General Mean Squared Error (MSE) across m samples

$$MSE = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2$$

The factor of $\frac{1}{2}$ cancels out neatly during derivative operations.

8. Backpropagation: Calculating Gradients via Chain Rule Backpropagation propagates error backward from output to input to calculate $\frac{\partial L}{\partial \theta}$.

Step 1: Gradient at Output ($\hat{y}$)

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = 0.4 - 1.0 = \mathbf{-0.6}$$

Step 2: Gradients for Output Layer Weights (v_1, v_2) and Bias (c)

$$\frac{\partial L}{\partial v_1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial v_1} = (-0.6)(h_1) = (-0.6)(1) = \mathbf{-0.6}$$

$$\frac{\partial L}{\partial v_2} = (-0.6)(h_2) = \mathbf{-0.6}, \qquad \frac{\partial L}{\partial c} = \frac{\partial L}{\partial \hat{y}}(1) = \mathbf{-0.6}$$

Step 3: Gradients for Hidden Layer Parameters (w_1, b_1)

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial \hat{y}} v_1 = (-0.6)(0.2) = -0.12 \frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial h_1} \cdot ReLU'(1) = (-0.12)(1) = -0.12$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z_1} x = (-0.12)(1) = \mathbf{-0.12}, \qquad \frac{\partial L}{\partial b_1} = \mathbf{-0.12}$$

9. Gradient Descent Parameter Update Update all parameters using learning rate $\eta = 0.1$ and rule $\theta_{new} = \theta_{old} - \eta \frac{\partial L}{\partial \theta}$:

0.5 **Hidden Layer Updates:**

- $w_{1,new} = 1.0 - (0.1)(-0.12) = \mathbf{1.012}$
- $w_{2,new} = 0.5 - (0.1)(-0.12) = \mathbf{0.512}$
- $b_{1,new} = 0.0 - (0.1)(-0.12) = \mathbf{0.012}$
- $b_{2,new} = 0.5 - (0.1)(-0.12) = \mathbf{0.512}$

0.5 **Output Layer Updates:**

- $v_{1,new} = 0.2 - (0.1)(-0.6) = \mathbf{0.26}$
- $v_{2,new} = 0.2 - (0.1)(-0.6) = \mathbf{0.26}$
- $c_{new} = 0.0 - (0.1)(-0.6) = \mathbf{0.06}$

10. Verifying Improvement (Forward Pass Step 2) Re-evaluate network predictions using updated parameter values at $x = 1$:

- $z_1 = (1.012)(1) + 0.012 = 1.024 h_1 = 1.024$
- $z_2 = (0.512)(1) + 0.512 = 1.024 h_2 = 1.024$
- New Prediction: $\hat{y}_{new} = 0.26(1.024) + 0.26(1.024) + 0.06 = \mathbf{0.5925}$
- New Loss: $L_{new} = \frac{1}{2}(0.5925 - 1.0)^2 = \mathbf{0.0830}$

Quantity	Before Update	After Update	Improvement
Prediction ($\hat{y}$)	0.4000	0.5925	Closer to target 1.0
Half-Squared Loss (L)	0.1800	0.0830	**Reduced by** 53.9%

11 & 12. Part C — Bias–Variance Trade-off 0.33 Underfitting

- Model capacity too low.
- E_{train} High, E_{test} High.
- High Systematic Bias.

0.33 Optimal Capacity

- Balanced complexity.
- E_{train} Low, E_{test} Low.
- Good Generalization.

0.33 Overfitting

- Capacity excessively high.
- $E_{train} \approx 0$, E_{test} High.
- High Variance.

Expected Generalization Error Decomposition

$$ExpectedError = Bias^2 + Variance + IrreducibleNoise$$

13. Part D — Classification: Sigmoid & Binary Cross-Entropy Shift task from regression to **Binary Classification** ($y \in \{0, 1\}$).

Step 1: Compute Logit to Probability Mapping via Sigmoid

$$p = \sigma(z) = \frac{1}{1+e^{-z}} \quad If z = 2.0: \quad p = \frac{1}{1+e^{-2}} \approx \mathbf{0.881} \quad (88.1\% probofClass1)$$

Step 2: Binary Cross-Entropy Loss (BCE)

$$L_{BCE} = -\left[y \log p + (1-y)\log(1-p)\right]$$

- If True Label $y = 1 L = -\log(p)$.
- If $p = 0.881 L = -\log(0.881) \approx \mathbf{0.127}$ (Low loss for correct confident prediction).

Task Type	Output Activation	Output Range	Loss Function
Continuous Regression	Linear / Identity ($f(z) = z$)	$(-\infty, \infty)$	Mean Squared Error (MSE)
Binary Classification	Sigmoid ($\sigma(z)$)	$(0, 1)$	Binary Cross-Entropy (BCE)
Multiclass Classification	Softmax (p_i)	$[0, 1]$, $\sum p_i = 1$	Categorical Cross-Entropy

14. Multiclass Classification: Softmax Categorical Cross-Entropy For K-class classification (e.g., MRI tumor diagnosis: Glioma, Meningioma, Pituitary):

1. Softmax Activation Function: Converts raw logits $z = [z_1, z_2, \ldots, z_K]^T$ into probabilities:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Numerical Example: Given network logits $z = [2.0, 1.0, 0.0]^T$:

- $e^{2.0} \approx 7.389, \quad e^{1.0} \approx 2.718, \quad e^{0.0} = 1.0 Sum = 11.107$
- $p = \left[\frac{7.389}{11.107}, \frac{2.718}{11.107}, \frac{1.0}{11.107}\right] = [\mathbf{0.665}, \mathbf{0.245}, \mathbf{0.090}]$

2. Categorical Cross-Entropy Loss: If True Class is Category 2 ($y = [0, 1, 0]^T$):

$$L_{CCE} = -\log(p_2) = -\log(0.245) \approx \mathbf{1.407}$$

15. Summary: Task vs. Output Layer Selection

Golden Rule Hidden layers use non-linear activations (e.g., ReLU) to learn representations; the output activation function is dictated strictly by the task type.

16. Core Equations Cheat Sheet 0.5

1. **Neuron Pre-activation:**

$$z = w^T x + b$$

2. **ReLU Activation:**

$$ReLU(z) = \max(0, z)$$

3. **Sigmoid Activation:**

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

0.5

4. **Softmax Function:**

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

5. **MSE Loss:**

$$MSE = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i - y_i)^2$$

6. **Gradient Descent Update:**

$$\theta \leftarrow \theta - \eta \nabla_\theta L$$

17. Final Picture: The End-to-End Learning Loop

$$\text{Input } x \longrightarrow ForwardPass(\hat{y}) \longrightarrow LossL \longrightarrow Backprop(\nabla_\theta L) \longrightarrow GradientUpdate(\theta)$$

Final Summary Takeaway

- **Forward Pass:** Generates model predictions.
- **Loss Function:** Quantifies discrepancy against ground-truth targets.
- **Backpropagation:** Uses calculus chain rule to attribute error gradients to every parameter.
- **Gradient Descent:** Updates parameters to iteratively minimize loss and improve predictions.

Deep Learning - Forward and Back Propagation

Refai Arafat Rossi

October 2026

1 Introduction

[Output Layer Parameters & Training]Output Layer Parameters and Backpropagation Understanding Trainable Weights v_1, v_2, c and Gradient Updates Master's Level Lecture Deck Department of Computer Science / Data Science October 10, 2026

height

1. Clarifying Output-Layer Parameters Core Misconception to Avoid Output-layer weights v_1, v_2 and bias c are **NOT** calculated or derived from input-to-hidden parameters (w_1, w_2, b_1, b_2).

0.5 **Hidden-Layer Parameters:**

- w_1, w_2: Input-to-hidden weights
- b_1, b_2: Hidden neuron biases
- Govern transformation: $x \to h$

0.5 **Output-Layer Parameters:**

- v_1, v_2: Hidden-to-output weights
- c: Output neuron bias
- Govern transformation: $h \to \hat{y}$

Takeaway v_1, v_2, c are independent, learnable model parameters. They are initialized randomly (or with small values) at the start of training and optimized via backpropagation.

2. Problem Setup and Given Initial Values Consider a simplified feedforward network architecture:

- **Input Features:** $x_1 = 3, \quad x_2 = -1$
- **Given Hidden Weights:** $w_1 = 1, \quad w_2 = 2$

- **Given Hidden Biases:** $b_1 = 1, \quad b_2 = 1$
- **Ground Truth Target:** $y = 1.0$

Hidden Layer Function Formulation:

$$h_1 = ReLU(w_1 x_1 + b_1), \qquad h_2 = ReLU(w_2 x_2 + b_2)$$

Architectural Note In a fully connected layer with 2 inputs and 2 hidden units, there are normally 4 input-to-hidden weights ($W \in R^{2\times 2}$). Here we use a 1-to-1 connection mapping for step-by-step clarity.

3. Step 1: Calculate Hidden-Layer Outputs (h_1, h_2) 0.5 Hidden Neuron 1 Computation Pre-activation:

$$z_1 = w_1 x_1 + b_1 = (1)(3) + 1 = \mathbf{4}$$

Apply ReLU activation:

$$h_1 = \max(0, z_1) = \max(0, 4) = \mathbf{4}$$

0.5 Hidden Neuron 2 Computation Pre-activation:

$$z_2 = w_2 x_2 + b_2 = (2)(-1) + 1 = \mathbf{-1}$$

Apply ReLU activation:

$$h_2 = \max(0, z_2) = \max(0, -1) = \mathbf{0}$$

Hidden Representation Output Vector:

$$\mathbf{h} = h_1$$

$\mathrm{h}_2 = \ 4$
0

4. Step 2: Where Output Parameters v_1, v_2, c Enter The output layer combines hidden neuron activations linearly for regression:

$$\hat{y} = v_1 h_1 + v_2 h_2 + c$$

Substituting our calculated hidden outputs ($h_1 = 4, h_2 = 0$):

$$\hat{y} = v_1(4) + v_2(0) + c = \mathbf{4v_1} + \mathbf{c}$$

0.55 **Key Observations:**

- v_1 scales the active feature ($h_1 = 4$).

- v_2 contributes 0 to prediction because $h_2 = 0$ (deactivated by ReLU).
- c shifts the baseline output.

0.45 Why Initialize Early? Forward propagation cannot complete without numerical values for v_1, v_2, c. We pick initial estimates to start the training loop.

5. Step 3: Forward Pass Loss Evaluation Let's set standard illustrative starting values:

$$v_1 = 0.2, \quad v_2 = 0.2, \quad c = 0.0$$

1. Compute Network Prediction ($\hat{y}$):

$$\hat{y} = 4(0.2) + 0(0.2) + 0 = 0.8 + 0 + 0 = \mathbf{0.8}$$

2. Compute Prediction Error (e):

$$e = \hat{y} - y = 0.8 - 1.0 = \mathbf{-0.2}$$

3. Evaluate Half-Squared Error Loss (L):

$$L = \frac{1}{2}(\hat{y} - y)^2 L = \frac{1}{2}(0.8 - 1.0)^2 = \frac{1}{2}(-0.2)^2 = \frac{1}{2}(0.04) = \mathbf{0.02}$$

6. Step 4: Backpropagation Gradients for Output Parameters Using the chain rule, differentiate loss $L = \frac{1}{2}(\hat{y} - y)^2$ with respect to output parameters:

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = 0.8 - 1.0 = -0.2$$

0.33 Gradient w.r.t. v_1 $\partial L \overline{\partial v_1 = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial v_1} = (\hat{y} - y) h_1 = (-0.2)(4) = \mathbf{-0.8}}$

0.33 Gradient w.r.t. v_2 $\partial L \overline{\partial v_2 = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial v_2} = (\hat{y} - y) h_2 = (-0.2)(0) = \mathbf{0.0}}$

0.33 Gradient w.r.t. c $\partial L \overline{\partial c = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial c} = (\hat{y} - y)(1) = \mathbf{-0.2}}$

7. Step 5: Gradient Descent Parameter Updates Apply parameter update rule $\theta_{new} = \theta_{old} - \eta \frac{\partial L}{\partial \theta}$ with learning rate $\eta = 0.1$:

- **Update v_1:**

$$v_{1,new} = 0.2 - 0.1(-0.8) = 0.2 + 0.08 = \mathbf{0.28}$$

- **Update v_2:**

$$v_{2,new} = 0.2 - 0.1(0.0) = 0.2 - 0.0 = \mathbf{0.20} \quad (\textit{Unchanged!})$$

- **Update c:**

$$c_{new} = 0.0 - 0.1(-0.2) = 0.0 + 0.02 = \mathbf{0.02}$$

Metric	Initial Pass	Updated Pass	Change
Prediction ($\hat{y}$)	0.8000	1.1400	Moved toward target 1.0
Loss (L)	0.0200	0.0098	**Reduced by** 51.0%

Insight on Inactive Neurons v_2 receives zero gradient update because its corresponding hidden neuron $h_2 = 0$. Inactive neurons transmit no error information back to their incoming weights.

8. Step 6: Verifying Updated Output Performance Re-evaluate forward propagation using new parameters ($v_{1,new} = 0.28, v_{2,new} = 0.20, c_{new} = 0.02$):

- Hidden Activations: $h_1 = 4, \quad h_2 = 0$
- New Prediction: $\hat{y}_{new} = (0.28)(4)+(0.20)(0)+0.02 = 1.12+0+0.02 = \mathbf{1.14}$
- Target $y = 1.0 NewErrore_{new} = 1.14 - 1.0 = +0.14$
- New Loss: $L_{new} = \frac{1}{2}(0.14)^2 = \mathbf{0.0098}$

9. Complete Architecture & Data Flow Diagram [x=2.2cm, y=1.3cm, ¿=stealth] [circle, draw, fill=blue!10, minimum size=0.9cm] (x1) at (0, 1) $x_1 = 3$; [circle, draw, fill=blue!10, minimum size=0.9cm] (x2) at (0, -1) $x_2 = -1$;
[circle, draw, fill=green!10, minimum size=0.9cm, align=center] (h1) at (1.5, 1) $h_1 = 4$; [circle, draw, fill=green!10, minimum size=0.9cm, align=center] (h2) at (1.5, -1) $h_2 = 0$;
[circle, draw, fill=red!10, minimum size=0.9cm] (y) at (3, 0) $\hat{y}$;
[-¿, thick] (x1) – node[above] $w_1 = 1, b_1 = 1$ (h1); [-¿, thick] (x2) – node[below] $w_2 = 2, b_2 = 1$ (h2);
[-¿, thick] (h1) – node[above right] $v_1 = 0.28$ (y); [-¿, thick] (h2) – node[below right] $v_2 = 0.20$ (y);
[draw=none] (c) at (3, -1.2) Bias $c = 0.02$; [-¿, dashed] (c) – (y);

Summary Equation

$$\hat{y} = v_1 \cdot ReLU(w_1 x_1 + b_1) + v_2 \cdot ReLU(w_2 x_2 + b_2) + c$$

10. Summary Takeaways for Students

1. **Parameter Separation:** Input-to-hidden parameters (w_i, b_i) determine feature representations (h_i). Output-layer parameters (v_i, c) dictate how those representations combine into final predictions ($\hat{y}$).

2. **Trainable Nature:** Output parameters are independent learnable quantities initialized prior to training; they are not derived mathematically from input weights.

3. **Gradient Masking by Activations:** When a neuron's activation is zero ($h_2 = 0$), its outgoing weight gradient becomes zero ($\frac{\partial L}{\partial v_2} = 0$), temporarily halting updates for that connection.

4. **The Core Learning Loop:**

   $$Inputs\mathbf{x} \longrightarrow Hidden\mathbf{h} \longrightarrow Output\hat{y} \longrightarrow LossL \longrightarrow Gradients\nabla L \longrightarrow Updates\theta_{new}$$